PROJET BIG DATA ELK MOVIES

Membre	Rôle
Sandrine YAO	Lead technique
Fride Audrey MOBOU	Chef de projet et Développeuse
Destiné	Développeur
Alexandre	Développeur

Tableau 1: Répartition des taches

Table des matières

LISTE DES TABLEAUX 2
LISTE DES FIGURES 3
I. Contexte, objectifs et périmètre 4
Périmètre fonctionnel 4
II. Architecture et environnement 4
Flux de données 4
III. Données et nettoyage (Feature 3) 5
III.1 Anomalies identifiées 5
Les anomalies qui ont été identifiée durant notre analyse sont les suivantes : 5
III.2 Règles de nettoyage appliquées 5
III.3 Impact du nettoyage 5
IV. Modélisation Elasticsearch et qualité des données (Feature 4) 6
Analyzer personnalisé 6
V. Requêtes, visualisations et lecture métier (Feature 5 & 6) 6
V.1.Types de requêtes 6
Les questions qui ont retenu notre attention nous ont permis de faire les requêtes suivantes : 6
V.2.Dashboard Kibana 7
Insights principaux 7
VI. Moteur de recherche Flask (Feature 8) 9
VII. Gestion de projet 9
Estimation des charges 9
VIII. Bilan, limites et perspectives 10
Points bloquants résolus 10
Limites identifiées 10
Axes d'amélioration 10
Conclusion 10

LISTE DES TABLEAUX

Tableau 1: Répartition des taches 1
Tableau 2: Services de la structure ELK 4
Tableau 3 : Types des données mappées 6
Tableau 4 : Planning Pocker simplifié 10

LISTE DES FIGURES

Figure 1 : Répartition des films par genre..... 7
Figure 2: Total des films avec date de publication, par genre et par année 8
Figure 3: Total des films avec date de publication, par genre, par année et ayant un budget et revenu > 0..... 8
Figure 4: Top 3 des films les plus populaires ayant un budget et revenu > 0..... 8
Figure 5 : Mini moteur de recherche ELK Movies..... 9

I. Contexte, objectifs et périmètre

Ce projet a pour objectif la conception d'une plateforme complète d'analyse de données cinématographiques à partir d'un dataset ; dans notre cas, nous avons pris comme source kaggle où nous avons utilisé les données fichiers movies.csv (662 083 films, 373 Mo). L'enjeu principal est de mettre en place une architecture ELK complète, reproductible et documentée, permettant l'ingestion de données brutes, leur nettoyage, leur modélisation et leur exploitation analytique via Elasticsearch et Kibana.

Périmètre fonctionnel

Le projet couvre l'ensemble de la chaîne de traitement des données cinématographiques :

- La création de l'architecture ELK de notre projet (elasticsearch-kibana-logstash)
- L'ingestion brute des données dans un index dédié (movies_raw)
- Le nettoyage et la normalisation vers un second index (movies_clean)
- La définition d'un mapping explicite avec analyzer personnalisé
- La création de 12 requêtes DSL analytiques (dont 5 requêtes booléennes)
- La réalisation d'un dashboard Kibana (8 visualisations)
- Le développement d'un moteur de recherche Flask connecté à Elasticsearch

II. Architecture et environnement

L'architecture retenue repose sur une stack ELK orchestrée via Docker Compose. Elle comprend quatre services interconnectés via un réseau interne (elk_network), avec des mécanismes de healthcheck garantissant un démarrage ordonné.

Service	Rôle principal
Elasticsearch	Stockage, indexation et recherche. Déployé en version 8.13.0, sans authentification, avec un seul nœud (suffisant pour usage local). Mémoire Java volontairement limitée.
Kibana	Visualisation et analyse. Configuré pour dépendre d'Elasticsearch, démarre uniquement lorsque celui-ci est disponible.
Logstash	Ingestion et transformation des données. Point d'entrée unique des données, accès en lecture seule au dossier data contenant movies.csv.
Flask API	Moteur de recherche REST et interface web. Connecte les utilisateurs à l'index Elasticsearch via une interface de recherche full-text.

Tableau 2: Services de la structure ELK

Flux de données

Le flux de traitement suit une logique linéaire permettant de comparer les données avant et après nettoyage :

movies.csv → Logstash → movies_raw + movies_clean → Kibana + API Flask

III. Données et nettoyage (Feature 3)

L'analyse du dataset movies.csv a mis en évidence plusieurs anomalies de qualité nécessitant un nettoyage structuré avant toute exploitation analytique. Les règles appliquées ont été définies en tenant compte de l'importance métier de chaque champ.

III.1 Anomalies identifiées

Les anomalies qui ont été identifiée durant notre analyse sont les suivantes :

- Types incorrects : valeurs numériques stockées en chaînes
- Champs multi-valeurs mal structurés
- Dates non typées
- Données manquantes ou nulles (ex : release_date absent dans ~9 % des films)
- Documents incomplets (6 supprimés après analyse)
- Budgets et revenus à zéro dans plus de 95 % des cas

III.2 Règles de nettoyage appliquées

Les champs critiques title et id sont considérés comme indispensables : les documents dépourvus de ces champs ont été supprimés. Le champ release_date, manquant dans environ 9 % des films, a été conservé avec une valeur égale à 0 afin d'éviter une perte excessive de données.

Concernant les valeurs nulles, les champs budget et revenue ont été requalifiés à -1 afin d'éviter toute interprétation erronée lors des calculs financiers. Les autres transformations que nous avons mis en place incluent :

- Conversion des types (string → float → integer/long)
- Parsing des dates (release_date)
- Transformation des champs multi-valeurs en tableaux
- Remplacement des valeurs manquantes (ex : -1 ou « N/A »)
- Suppression des documents invalides

III.3 Impact du nettoyage

Le nettoyage a permis d'obtenir un index movies_clean exploitable, offrant des requêtes analytiques fonctionnelles, des agrégations pertinentes et une amélioration significative de la qualité des données disponibles pour l'analyse.

IV. Modélisation Elasticsearch et qualité des données (Feature 4)

L'index movies_clean repose sur un mapping explicite définissant précisément le type de chaque champ. Les champs numériques ont été convertis en types integer, float ou long selon leur nature et leur plage de valeurs. Les champs textuels sont scindés entre text et keyword afin de concilier recherche full-text et agrégations.

Type de champ	Mapping Elasticsearch
Champs textuels	Type text avec analyzer personnalisé
Champs catégoriels	Type keyword
Dates	Type date
Données financières	Type long
Données numériques	Types integer ou float

Tableau 3 : Types des données mappées

Analyzer personnalisé

Un analyseur franco-anglais (french_english_analyzer) a été implémenté pour améliorer la pertinence des recherches textuelles. Il combine :

- Lowercasing et normalisation des caractères (suppression des accents)
- Filtrage des mots vides (stop words français et anglais)
- Stemming (racinisation des mots)

V. Requêtes, visualisations et lecture métier (Feature 5 & 6)

Douze requêtes analytiques Elasticsearch ont été développées pour répondre à des questions métier concrètes, telles que l'identification des films les plus populaires, l'analyse de la durée moyenne par genre, la rentabilité des productions ou la détection de flops commerciaux. Certaines anomalies (ex : budgets irréalistes) ont nécessité des ajustements via des filtres supplémentaires.

V.1.Types de requêtes

Les questions qui ont retenu notre attention nous ont permis de faire les requêtes suivantes :

- Identification de films populaires
- Détection de flops commerciaux
- Analyse par genre ou langue
- Recherche full-text avec highlight
- 5 requêtes booléennes complexes (must, should, filter, must_not)

V.2.Dashboard Kibana

Le dashboard Kibana regroupe 8 visualisations permettant une exploration intuitive des données par langue, genre, période de sortie et popularité :

- Répartition des films par genre
- Évolution de la production cinématographique dans le temps
- Analyse des budgets et revenus
- Classement des films les plus populaires
- Comparaison des performances financières

Insights principaux

Nous constatons à travers notre dashboard que le genre dominant de notre base cinématographique est le drama.

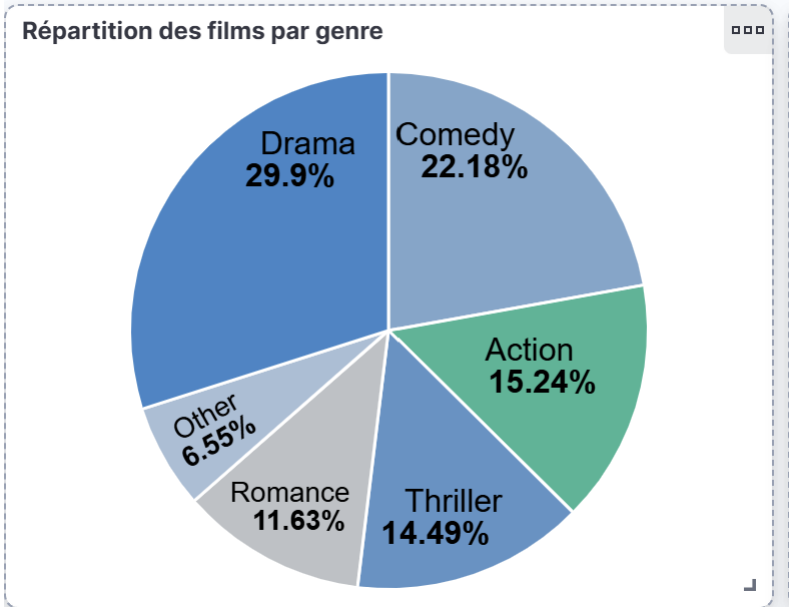

Figure 1 : Répartition des films par genre

De plus, nous pouvons noter que 1,63% (soit 9830/602562) des films de notre base ont un budget et un revenu positif

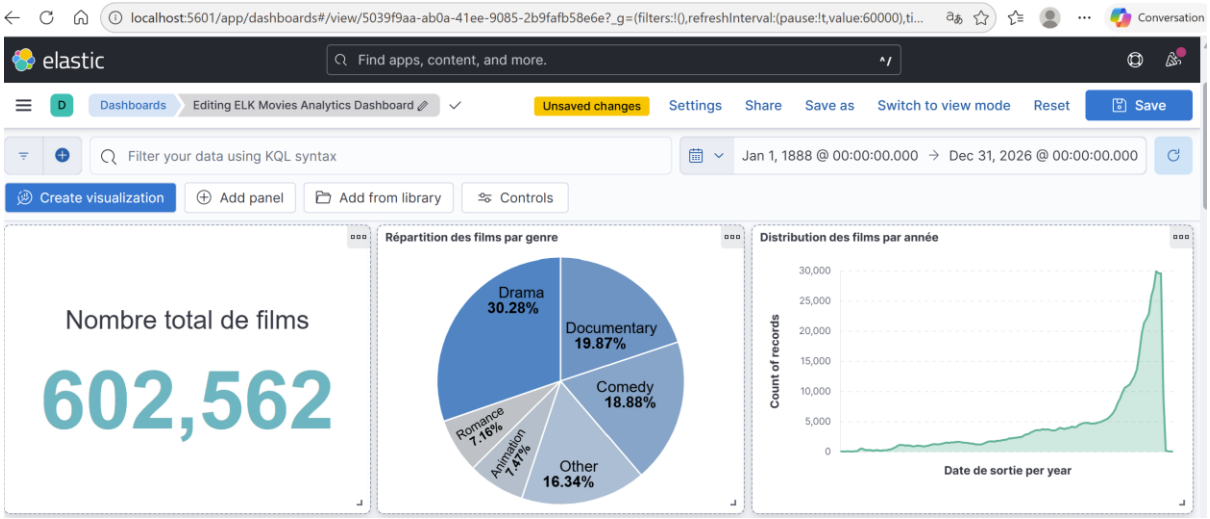

Figure 2: Total des films avec date de publication, par genre et par année

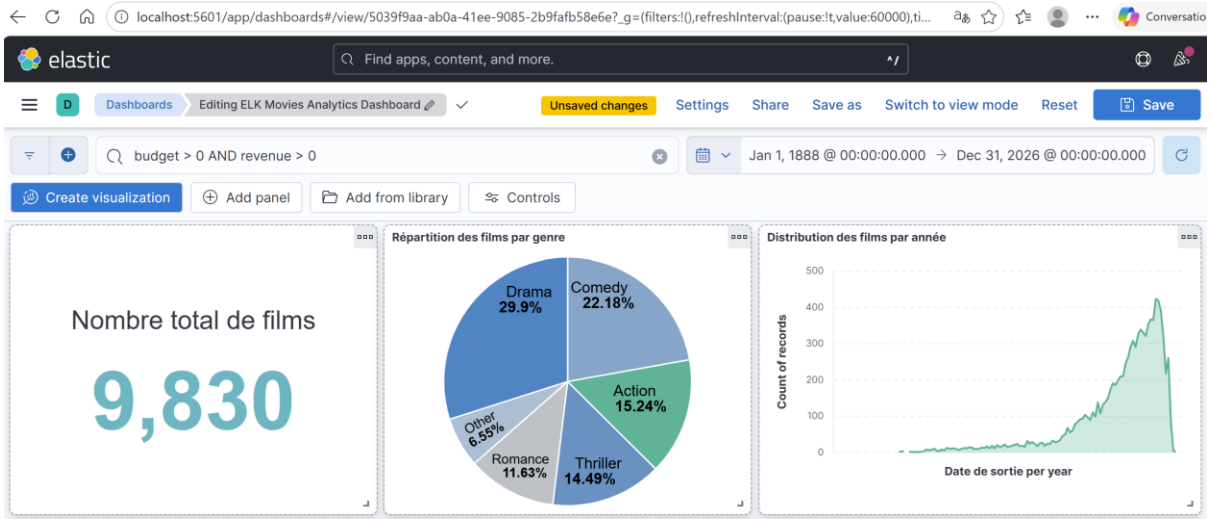

Figure 3: Total des films avec date de publication, par genre, par année et ayant un budget et revenu > 0

Parmi ces films c'est The Northman qui est le plus populaire

Films	Popularité
The Northman	3,669.153
Fantastic Beasts: The Secrets of Dumbl	3,456.961
The Lost City	3,190.602

Figure 4: Top 3 des films les plus populaires ayant un budget et revenu > 0

VI. Moteur de recherche Flask (Feature 8)

Un mini moteur de recherche connecté à Elasticsearch a été développé en Flask, permettant de réaliser des recherches full-text avec filtres simples. Cette interface applicative démontre l'usage concret des données nettoyées dans un contexte métier.

L'API expose un endpoint REST de recherche, avec support du highlight Elasticsearch. Note : le moteur Flask n'est pas encore intégré dans Docker Compose, ce qui représente un axe d'amélioration identifié.

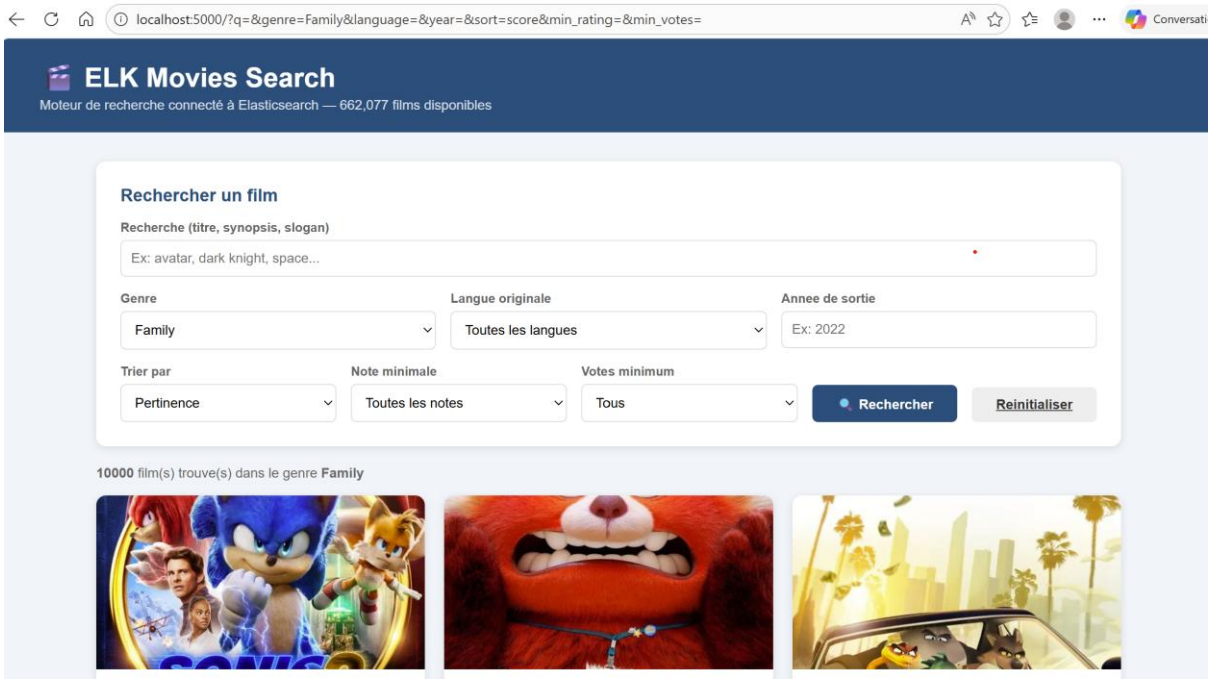

Figure 5 : Mini moteur de recherche ELK Movies

VII. Gestion de projet

Le projet a suivi une organisation Gitflow structurée, avec des branches main et dev protégées, un développement par feature et l'utilisation systématique de Pull Requests et de code reviews pour garantir la qualité du code.

Estimation des charges

Un planning poker basé sur la suite de Fibonacci a été utilisé pour évaluer les tâches. Les fonctionnalités les plus complexes ont été le nettoyage des données et le moteur de recherche Flask. La répartition des tâches a été équilibrée entre les membres de l'équipe.

Membre	Features	Points total
Sandrine (lead)	F2 *Ingestion brute* + F7 *Documentation* + F3 *Normalisation* + F6 *Dashboard Kibana*	3 + 3 + 3 + 8 = **17**
Audrey	F1 *Bootstrap stack* + F7 *Documentation* + F3 *Anomalies* + F6 *Dashboard Kibana*	3 + 3 + 2 + 8 = **16**
Destiné	F3 *Nettoyage (types + date)* + F6 *Dashboard Kibana* + F5 *Requêtes DSL*	3 + 8 + 5 = **16**
Alexandre	F4 *Mapping qualité* + F8 *Moteur de recherche*	5 + 8 = **13**

Tableau 4 : Planning Pocker simplifié

VIII. Bilan, limites et perspectives

Points bloquants résolus

- Problèmes de mapping Elasticsearch (conflits de types)
- Limites des types numériques (overflow long)
- Contraintes de volume des données (662 083 documents)
- Incompatibilités API entre les versions de services

Limites identifiées

- Données manquantes persistantes (dates, budgets, revenues)
- Encodage imparfait sur certains titres
- Moteur Flask non dockeré (déploiement incomplet)

Axes d'amélioration

- Conténérisation complète de l'API Flask dans Docker Compose
- Amélioration du scoring de recherche (boosting, BM25)
- Enrichissement des données (API TMDB, IMDb)
- Automatisation du déploiement (CI/CD)
- Intégration de nouveaux analyzers spécialisés

Conclusion

Ce projet démontre la capacité de l'équipe à concevoir une plateforme data complète avec le stack ELK, couvrant l'ensemble de la chaîne de valeur :

Ingestion → Transformation → Modélisation → Analyse → Visualisation → Exploitation

Les choix techniques et méthodologiques ont été guidés par des contraintes de qualité des données et de reproductibilité. Il met également en évidence les enjeux réels liés à la qualité des données et à leur exploitation dans un contexte analytique. Des améliorations futures pourraient inclure l'enrichissement des données, l'optimisation des performances de recherche et une conteneurisation complète de la solution.